

Part A - Experience Beyond the CV

- 1. Describe one technical project you personally participated in. What was the goal, what was your role, and what exactly did you implement yourself?**

I worked on a recruiting system built with Laravel and PHP, using Blade templates and Tailwind CSS. Recruiters use it to manage jobs, candidates and employees.

I did many different tasks on this project. I started by working inside their existing template system, so first I had to read their code and understand how their Blade views and layouts are built. Then I was building my own templates.

I fixed visual bugs things like wrong menu highlighting, content not aligned in the candidate list, and names not showing in the dashboard.

Later I also implemented completely new features. The latest new feature I built was the AI candidate summary and Job Match score. I built the candidate card that shows a short 4-6 line summary of the CV and a match score against the job, so the recruiter can see who to look at first.

I also did employee referral flow. This is a page where an employee sees the company's open jobs, and logic that gives that employee credit if the person they referred gets hired.

- 2. Give an example of a technical problem you had to debug. What was broken, how did you check it, and what eventually fixed the problem?**

On the dashboard there is a widget that shows a list of candidates. Some rows showed only empty space instead of the candidate name.

To fix it, I opened the file and looked at how the name is printed. The code was `(firstname . ' ' . lastname)`. If both fields were empty in the database, the result was just an empty string. So nothing was shown, and there was no error that is why it was easy to miss. The fix was small: I added a fallback, `(? : ' Unknown ')`. Now if a candidate has no name, the widget shows "Unknown" instead of blank space. I tested after with a candidate that had no first name and no last name, to be sure the row shows correctly.

3. Which technical area do you want to improve next, and how exactly will you study it?

Right now I have a basic understanding of a lot of things, but I would not say that any of them I am not an expert. I want to improve in all of them and, most of all, get more real experience by actually using them in a work environment.

When I want to learn something new, I usually start by reading the documentation, or I watch some videos about the subject to get the general idea first. But in my opinion the best way to learn is by doing. I really understand something when I build it myself, make mistakes, fix them, and then do similar projects again. The repeating part is important for me.

4. What are your strongest programming areas? Mention the direction, language, or technology where you feel most confident.

I feel my strongest part is the backend. I am comfortable working with PHP and Laravel models, controllers, queries, Blade templates and so on. That is where I have worked the most.

I have some experience with the frontend - React, Tailwind CSS and JavaScript.

Part B - Git Thinking

5. What makes a commit message good? Give three examples: bad, acceptable, and good

A good commit message says what was changed and why, so that another developer or me after a few months can understand it without opening the code.

Bad - fixed bug.

Acceptable - fixed candidate name in dashboard.

Good - From my experience, when you work in a team every bug or feature already has its own task code, so the commit message should start with that code and then say shortly what was done and where.

For example

Task01 - Highlight Employees tab on Add Employee

The page passed 'job' as the menu value to the layout, so the top navigation highlighted Jobs while the user was in the Employees section. Now it passes the correct value.

Other pages in the same folder were checked for the same mistake.

6. A colleague pushed code that broke the main branch. What would you do first?

First I would write to the team, so nobody pulls the broken main. I would also write to the colleague who pushed it, because he knows his code best.

7. Which files should usually not be committed to a repository? Give examples for a web application project

Probably .env file first, because they contain sensitive information like passwords and API keys. Then everything in .gitignore and /vendor.

Part C - Docker / Environment Understanding

8. Explain what Docker is and why a development team may use it.

Docker lets every project run in its own container with its own PHP and database versions, so you can work on many projects on one computer simultaneously without problems with version conflicts.

It is also very popular, so for most things you need PHP, MySQL and so on. There are already made images, and you do not need to set everything up from zero yourself.

9. What is the difference between an image and a container?

An image is like a template - it is built once and only holds the files and settings. It does not run by itself.

A container is that image actually running.

10. What is docker-compose.yml used for?

docker-compose.yml is a file where you write all the containers the project needs PHP, nginx and MySQL with their settings, so instead of starting each one separately you can run (docker compose up) and everything starts together.

11. If a web app works on one developer's computer but not on another, how can Docker help? What problems may still remain?

Probably it is a setup problem with a different PHP version, wrong .env or missing extension.

Docker helps because everybody runs the same containers with the same PHP, extensions and database. But you need to setup .env file yourself locally because it's not on git.

Part D - Backend / Framework Thinking

12. Why do developers use a framework such as Laravel or Symfony?

Frameworks help to make processes easier and speed up development. Things like routing, database queries and logins take a lot of time if you write everything from zero, but frameworks are helping you to do them faster and in a more universal way.

They also help with the project structure, because everyone is following the same structure instead of each developer organising the files in their own way.

13. What must always be validated on the backend, even if the frontend already performs validation?

Everything that comes from the user must be validated on the backend for security reasons, because the frontend is quite easy to manipulate.

A good example is my G1 project: I changed a dropdown value to (hacked) in devtools and submitted the form. The server refused it and saved (screenshots 05 and 06).

14. What is the difference between authentication and authorization?

Authentication is checking who you are - the login with email and password.

Authorization is checking what you are allowed to do.

15. What should an API return when validation fails? Provide an example response structure.

When validation fails, the API should return status 422 with a message and the errors grouped by field name, so the frontend can show message next to the right input.

```
{
  "message": "Validation failed",
  "errors": {
    "email": ["The email format is not valid."]
  }
}
```

Part F - Web Application / Product Thinking

Scenario: You are helping to build a web platform where users upload media files, the platform processes them in the background, and users later download the result

16. Which frontend screens or states does the user need?

Screens

- Login / register
- Upload screen - choose or drag the file, see formats and what size are allowed
- File list - all uploaded files
- Result screen - preview and download button

17. Which backend entities/tables would you expect?

- users - users - id, name, email, password
- files - user_id, original name, the path where the file is stored, size, status, uploaded_at
- results - file_id, type or format of the result, path, size, status, error message, finished_at, expires_at

18. Which background jobs may be required?

- Processing the uploaded file - convert or compress it, and make a preview or thumbnail
- Checking the file - virus scan and a check if that it is correct format
- Sending the user an email or a notification when the result are ready
- Cleanup - deleting expired results and files that are left from failed or unfinished uploads
- Retrying failed jobs, couple of times, so one broken file is not staying in the queue forever

19. What could go wrong if two users upload files at the same time?

If both users upload a file with the same name, one can overwrite the other, so every file must be saved with a unique generated name, and two big files processed at the same time could overload the server.

20. Mini Task C1 - Read and Explain docker-compose.yml

What each service does?

App - this is the PHP application. It is built from the Dockerfile in the project folder, not taken from a ready image. The project folder is synchronized inside, so when I change code on my computer, it also changes it in the container.

Web - this is nginx, the web server. It takes the requests from the browser, gives back static files and sends the PHP requests to the app container. It also has its own config file from the nginx folder.

Db - this is MySQL, the database. On the first start it makes the database `internship_test` and the user `app`.

Which ports are exposed?

web - 8080:80, so the site opens on `localhost:8080`.

db - 3306:3306, MySQL is not only inside Docker but also opens on the computer.

app - no ports

Where the data is stored?

The database data is in a volume called `db_data`, mounted to `/var/lib/mysql`. That is important, because the data stays there even if the container is deleted and remade. The code is not in a volume, it is just the project folder from the computer synchronized into the containers.

3 problems are

- Passwords have two problems. First - they are written in the compose file, which goes to Git, so anyone who can access repository can read them. They should come from the `.env` file. Second - `secret` and `root` are weak passwords. A password should be longer with upper and lower case letters and numbers.
- The database port should not be open. The app talks to the database inside docker, so `3306:3306` is not necessary. On the server it would mean that MySQL is reachable from the internet, and then only be protected by password.

- `depends_on` is not waiting for the database. It is waiting only until the container is started, not until MySQL can accept connections, so the app could crash on the first start. It needs a healthcheck, or the app has to try the connection again.

21. Mini Task C2 - Debugging Scenario

A developer says: "Container starts, but the application cannot connect to MySQL."

List at least 6 things you would check, in order.

1. First I would look at the error. "Connection refused" and "Access denied" are two different problems. Refused means the app cannot reach MySQL at all. Access denied means it reached it, but the user or password is wrong. So already from this I could decide where to look.
2. Then I would check if the db container is really running, with `docker compose ps`, and look at docker compose logs in db. Could be that MySQL crashes and the container just restarts all the time. On the first start it needs some time to make the database.
3. Then I would check `DB_HOST` in `.env`. This is a common mistake. If it says `localhost`, the app is looking for MySQL inside its own container. In docker it has to be the service name, so `DB_HOST=db`.
4. Then the port and the passwords. The port has to be matching, standard for docker is 3306. And the database name, user and password in `.env` have to be the same as in `docker-compose.yml`.
5. Then the old volume. MySQL makes the user and the password only on the first start, when the volume is still empty. So if someone changed the password later, the database still has the old one. Then the volume has to be deleted and made again.
6. Then I would check if the app really got the updated settings. After changing `.env` the container has to be recreated, not only restarted. In Laravel you also have to run `php artisan config:clear`, because the old config can be cached.
7. Then I would try to connect to the database from inside the app container. If from there it works, then the problem is in the app config and not in docker.
8. And lastly, the timing. `depends_on` waits only until the container is started, not until MySQL can take connections. So on the first start the app can try too early. If it works after a restart, then this was the reason.

22. Mini Task D1 - Find Problems in PHP Code

Problems:

1. **SQL injection.** The values go straight into the query text, so a quote in the name can change the whole query and read or delete data. This is a huge problem.
2. **The role comes from the user.** Anyone can send (role=admin) and make themselves an admin. The role must be set by the server.
3. **Nothing is validated.** The email is not checked, the name can be an empty string, and the text length is not limited.
4. **Fields are read without checking they exist.** If the form does not send a role, PHP throws a warning for the missing key.
5. **The result is not checked.** If the insert fails, for example a duplicate email, reports success anyway.
6. **The function mixes logic and output.** Creating a user and printing a message are two different jobs; the function should only return the result.

My code:

```
function createUser(PDO $conn, array $input): int
{
    $name = is_string($input['name'] ?? null) ? trim($input['name']) : '';
    $email = is_string($input['email'] ?? null) ? trim($input['email']) :
    '';

    // fixed role, not taken from the user
    $role = 'user';

    if ($name === '') {
        throw new InvalidArgumentException('Name is required.');
```

```
    }
    if (mb_strlen($name) > 255) {
        throw new InvalidArgumentException('Name can be at most 255
characters.');
```

```
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new InvalidArgumentException('Email is not valid.');
```

```
    }

    $sql = 'INSERT INTO users (name, email, role)
           VALUES (:name, :email, :role)';

    $stmt = $conn->prepare($sql);

    try {
        $stmt->execute([
            ':name' => $name,
            ':email' => $email,
            ':role' => $role,
        ]);
    } catch (PDOException $e) {
        // 23000 = unique index on email
        if ($e->getCode() === '23000') {
            throw new RuntimeException('This email is already
registered.');
```

```
        }
        throw $e;
    }

    return (int) $conn->lastInsertId();
}
```

What I fixed:

- **SQL injection** - the values go into the query as parameters, not as text.
- **Role** - set in the code, never taken from the form.
- **Validation** - name and email are checked before the insert, and each problem has its own message.
- **Missing or wrong fields** - anything missing or not a string becomes an empty string, so the code cannot break.
- **Failed insert** - a duplicate email gives a clear message instead of a raw database error.
- **No output** - the function returns the new user's id, so the page decides what to show.